

哨兵 × jev-decision-service 集成验收报告

把垂类决策头正式接入决策框架：接口、三层路由、留出集实测与阈值权衡

1 本次交付：把哨兵决策头正式接进 jev 框架

之前哨兵是一个独立服务（自带决策层、路由、看板），jev 框架是另一条线。本次交付把两条线合成一个系统：决策层复用 jev 的校准、策略、引擎与 HTTP 契约，哨兵只提供它缺的那一块——结构化传感器到概率分布的模型。

7 决策问题（1 Noul + 2 Choice + 1 Score + 3 策略桥）	3 层 路由：决策头 → 生成式 → 规则兜底	88.3% 自动化率（留出集实测）	0% 故障漏放率
--	----------------------------	----------------------	-------------

1.1 接入方式：不改框架一行代码

框架提供（复用）	哨兵提供（新增）
calibration.py 温度缩放 / 先验修正 / 分布漂移检测 / 稀有类告警	sentinel_heads.py 三个决策头的训练与推理（torch + GPU）
policy.py 确定性路由 + OOD / 稀有类 / 未标定拦截	make_evaluator() 把概率分布翻译成框架的答题卡
engine.py DecisionEngine（warnings 合并、记忆、幂等、trace）	industrial_profile() 工业维护版问题集（沿用 policy 认识的问题 id）
server.py / service.py /v1/decide、/v1/replay 路由与错误码	SentinelProvider 按输入模态分派三层 + 降级兜底
state_builder.py 结构化特征校验、模态识别、OOD 分数	看板 + /v1/decide/batch + /v1/eval/dataset（一键跑留出集）

接入点是 StructuredHeadProvider(evaluator=..., questions=...)——框架本来就把这个口子留好了（类注释写着「for a trained MLP/torch/sklearn decision head and a custom QuestionProfile」）。我们只填这个口子，没有 fork、没有 monkey patch。

2 工业版问题集：沿用框架的问题 id

框架的默认问题集是对话路由（意图、要不要澄清、是否授权副作用...）。我们换成工业维护语义，但刻意保留 policy 认识的问题 id，因此策略层一行都不用改，它那五条保护继续生效。

问题 id	类型	工业语义	来源
fault	Noul	该设备当前是否处于故障状态？	决策头（AI4I Machine failure）
fault_type	Choice(6)	故障类型：无故障 / 工具磨损失效 / 散热 / 功率 / 过载 / 随机	决策头（AI4I Failure Type）
risk_level	Score(4)	维护优先级：观察 / 计划 / 尽快 / 立即	决策头（派生标签，provenance=derived_rule）
needs_evidence	Noul	是否需要调取维修手册或历史工单	策略桥：确认有故障且类型判不准时为高
needs_clarification	Noul	传感器读数是否可疑、需现场先确认	策略桥：五项物理量程校验
allows_side_effect	Noul	是否足够可靠、可自动派工单	策略桥：置信度门控
next_action	Choice(5)	下一步动作（判正常 / 查证据 / 追问 / 开单 / 转人工）	策略桥：置信度三档 + 阻尼分布

后四项是策略桥：把已标定的置信度翻译成框架能消费的信号。它们的定义都在代码里显式写出，便于审计；而 fault / fault_type / risk_level 三项是真模型输出。

3 三层路由：实测可升级、可降级

层	实现	触发条件	实测
System One 决策头	哨兵 MLP(64,32)×3 @ GB10 GPU	有结构化特征且置信度达标	0.49 ms/条 (p50) , 置信度 0.999
System Two 生成式	本地 Qwen3-8B (ollama)	决策头置信度 < floor (默认 0.80)	16 秒, route=generative
兜底 规则	框架自带 RuleBasedProvider	上游 ProviderError	标记 degraded

3.1 升级链路实测 (把 floor 调到 0.995 强制触发)

观测项	结果
route / route_reason	generative / fast_head_low_confidence
provider	local-generative
端到端耗时	16 秒
生成式输出	fault=yes 0.95 , 但 fault_type=No Failure 0.92、next_action=answer_from_context
最终动作	ask_clarification (原因 : uncalibrated_probability)

两点值得注意：

① 生成式输出自相矛盾——一边说「是故障 95%」，一边判「无故障类型 92%」并建议直接回答。框架的 OpenSourceProvider 不做跨问题一致性约束；我们的决策头做了 (见第 6 节)，这是实打实的差异。

② 升级后仍不会自动处置——生成式的概率是 uncalibrated，策略层 require_calibration_for_auto 会拦住。所以升级层的定位是给人工提供第二意见，不是提升自动化率。这个行为是安全正确的，但要如实说明。

工程建议：升级层单次 16 秒，不应同步阻塞报警通道。生产形态应为「决策头同步出结果 → 低置信事件投递到异步队列 → 生成式结果回来后刷新复核项」。本服务默认关闭升级 (fast-only)，需要时用 ESCALATE=1 开启。

4 真实留出集端到端评测

评测走完整的框架链路 (state_builder → 决策头 → calibration → policy → engine)，数据是 AI4I 2020 的留出测试集 (训练未见过的 20%，n=1000)，由服务端内置端点 /v1/eval/dataset 执行。

4.1 主指标 (urgent_score = 2.5)

指标	数值	说明
自动化率	88.3%	无需人工介入即处置 (判正常归档 / 自动开工单)
自动部分准确率	100.0%	自动化部分与真值一致的比例 (883 条)
故障漏放率	0.0%	真值故障却被判定为正常 (最危险的一类错误)
正常设备误转率	9.1%	健康设备被转人工，属于可接受的成本
人工 / 查证占比	11.7%	117 条，其中 74 条因达到「立即级」强制转人工
引擎延迟	p50 0.494 ms / p95 0.525 ms	含 7 个决策 + 校准 + 策略判断
吞吐	2,002 条/秒	单进程、逐条走完整框架

4.2 动作 × 触发原因 交叉表 (一眼看清每条为什么这么走)

最终动作 ← 首个原因码	条数
answer_from_context ← policy_route (高置信判正常，自动归档)	882
human_review ← high_risk_or_urgent (达到「立即级」，强制人工)	74

最终动作 ← 首个原因码	条数
human_review ← side_effect_not_authorized (置信度 0.8~0.9 档, 落人工)	26
retrieve_evidence ← side_effect_not_authorized (0.8~0.9 档, 先查依据)	14
human_review ← rare_class_support_insufficient (命中 TWF/RNF 少数类)	3
draft_suggestion ← policy_route (高置信判故障, 自动开单)	1

中间档 (0.80 ~ 0.90) 只有 14 条——说明决策头的置信度分布很两极化：要么很确定，要么直接落人工。这解释了为什么「先查证据」这一档很少触发。

5 阈值扫描：自动化率与安全的拐点在哪

唯一可调的旋钮是 policy 的

urgent_score (严重度期望值达到多少就强制人工)。同一批留出集、同一套参数，只动这一个旋钮：

urgent_score	自动化率	自动部分准确率	故障漏放率	正常设备误转率	人工数
1.6	39.8%	100.0%	0.0%	59.1%	602
2.0	47.9%	100.0%	0.0%	50.7%	521
2.5 (默认)	88.3%	100.0%	0.0%	9.1%	117
3.0	92.7%	100.0%	0.0%	4.5%	73
3.05	94.1%	99.7%	10.0%	3.4%	59

拐点在 3.0 与 3.05 之间，而且是个悬崖：阈值再放宽一档，自动化率只多 1.4 个百分点，故障漏放率却从 0% 跳到 10% (3 条真实故障被当成正常归档)。这类「多赚一点自动化、赔掉安全底线」的区间，正是必须靠留出集量化、而不能凭感觉设阈值的原因。

6 安全保护逐条实测

保护	触发方式	动作	触发原因码
OOD 分布外	ood_score = 0.9	human_review (拦截)	out_of_distribution_requires_review
部署分布漂移	channel_metadata.distribution_id = line-B	human_review (拦截)	distribution_shift_requires_recalibration
同分布 (对照)	distribution_id = 标定分布名	answer_from_context (放行)	policy_route
传感器读数可疑	rpm=0 且过程温度低于环境温度	ask_clarification	clarification_needed
少数类命中	模型判出 TWF / RNF (训练样本 46 / 18 条)	human_review (拦截)	rare_class_support_insufficient
先验修正	channel_metadata.deployment_positive_rate = 0.5	概率由 0.0027 → 0.0774	由框架 apply_profile 应用
正常工况 (对照)	—	answer_from_context (放行)	policy_route

「诚实标注」的代价变成了收益：我们把 label_provenance 如实填为 derived_rule (严重度是规则派生)，框架把它作为警告回传，而不再像 0.1.3 早期那样把整套系统拦死——这正是 0.1.4 修好的语义。

7 接入过程中发现的框架级问题（供团队评估）

#	问题	位置	影响与建议
1	升级失败被静默掩盖：TieredProvider 捕获 ProviderError 后设置了 last_route=fast_degraded，但紧接着的收尾语句又把它覆盖成 fast / fast_head_accepted，且 degraded 标志未被上层读取	provider.py · TieredProvider.evaluate	上层无法区分「正常走快头」与「升级失败退回」。本次在包装层 SentinelProvider 里读取 degraded 后纠正；建议框架在降级分支直接 return
2	先验修正作用于**所有** noul 型预测，而非仅二元故障概率	calibration.py · apply_profile → _calibrate_probability	实测 deploy_rate=0.5 时：fault 0.0027→0.0774（预期），但 needs_clarification 0.05→0.62、allows_side_effect 0.90→0.996（非预期）。若先验偏离更大，会因 0.62 逼近 0.70 阈值而误触发追问。建议_先验修正只作用于显式声明的目标问题
3	score 类型只校验取值范围，不校验概率键名	provider.py · _parse_jev_answer / _normalize_raw_answers	生成式实测返回 {'0':0.85,'1':0.1} 而非 {'观察':...}，键名语义丢失，下游按标签取值的逻辑会失效。建议对 score 也校验键集合

这三条都不是阻断性问题，我们已在包装层规避，框架代码保持原样。是否向上游修改由团队决定——我们倾向提出而不擅自改，以保持框架副本与团队主线一致。

8 已知局限与下一步

局限	说明	下一步
单一数据集	决策头在 AI4I 2020 上训练；跨数据集评测（5 个工业子领域）是在统一框架下做的，尚未把这套集成栈逐个跑过	把 multi_dataset_bench 的 5 个 adapter 接到本服务上，出跨领域版验收
少数类仍弱	fault_type 宏平均 F1 仅 0.211（TWF 46 条、RNF 18 条）	用 StepFun 合成少数类样本，或改用类平衡损失重训
升级层未标定	生成式概率 uncalibrated，策略层不敢自动处置	这是安全设计；若要让升级层参与自动化，需对其做温度标定
severity 为派生标签	risk_level 的 0~3 档由规则派生，非真实标注	与领域专家或 StepFun 复核一版标签，再重训

哨兵 EdgeSentinel · 集成验收 · 数据均来自节点 gx10-902c 实测